

Task B: BankAccount

Data Structures & Algorithms · Universidad de Dagupan

bank_account.py

```
class BankAccount:
    def __init__(self, owner, balance):
        if balance < 0:
            raise ValueError("Opening balance cannot be negative")
        self.owner = owner
        self._balance = balance

    def get_balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self._balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive")
        if amount > self._balance:
            raise ValueError("Insufficient funds")
        self._balance -= amount

    def __str__(self):
        return f"{self.owner}: {self._balance:.2f}"
```

Sanity Check Results

Run manually (no test_bank_account.py was provided) covering opening balance, deposit, withdraw, and the three rejection cases from the spec.

```
1500
Juan: 1500.00
Juan: 1750.50
Juan: 750.50
pass: negative opening balance rejected
pass: zero deposit rejected
pass: overdraw rejected
```

Spec Coverage

Operation	Requirement	Status
__init__(owner, balance)	Rejects negative opening balance	Pass
get_balance()	Returns balance; callers never touch _balance	Pass
deposit(amount)	Rejects zero or negative	Pass
withdraw(amount)	Rejects overdraw	Pass

<code>__str__()</code>	Two decimal places, e.g. Juan: 1500.00	Pass
------------------------	--	------